

CHRIST (Deemed to be University), Bengaluru  
Department of Computer Science

## Sprint II: Design and Development Document

Course: CSC481-5, Computer Science Project  
Semester V

# ThinkStack

*Packaging, Distribution and Verified Delivery*

### Submitted by

| Sl. No. | Student Name  | Register No. |
|---------|---------------|--------------|
| 1.      | Aditya Mehta  | 2440204      |
| 2.      | Jitvan Chadha | 2440222      |
| 3.      | Ritesh K R    | 2440233      |

**Guide:** Dr. New Begin  
**Class:** 5BSC CS  
**Repository:** [github.com/get-thinkstack/ThinkStack](https://github.com/get-thinkstack/ThinkStack)

July 2026

## Contents

|          |                                                                   |          |
|----------|-------------------------------------------------------------------|----------|
| <b>1</b> | <b>Sprint Overview</b>                                            | <b>2</b> |
| 1.1      | Period and scope . . . . .                                        | 2        |
| 1.2      | Sprint goal statement . . . . .                                   | 2        |
| <b>2</b> | <b>Design Problem: The Unprepared Machine</b>                     | <b>2</b> |
| <b>3</b> | <b>Design of the Distribution System</b>                          | <b>2</b> |
| 3.1      | Packaging architecture . . . . .                                  | 2        |
| 3.2      | Release channel design . . . . .                                  | 3        |
| 3.3      | Release guardrails . . . . .                                      | 3        |
| <b>4</b> | <b>Implementation</b>                                             | <b>3</b> |
| 4.1      | Phase 1: packaging and delivery, 7 to 24 July . . . . .           | 3        |
| 4.2      | Phase 2: quality infrastructure, 25 to 29 July . . . . .          | 4        |
| 4.3      | Phase 3: release, failure and correction, 29 to 30 July . . . . . | 4        |
| <b>5</b> | <b>Root Cause Analysis</b>                                        | <b>4</b> |
| 5.1      | Why the application did not start . . . . .                       | 4        |
| 5.2      | Further defects found in the same investigation . . . . .         | 4        |
| 5.3      | Corrections applied . . . . .                                     | 5        |
| <b>6</b> | <b>Design of the Verification System</b>                          | <b>5</b> |
| 6.1      | Dependency audit . . . . .                                        | 5        |
| 6.2      | An instructive packaging defect . . . . .                         | 6        |
| <b>7</b> | <b>Testing and Results</b>                                        | <b>6</b> |
| 7.1      | Automated verification . . . . .                                  | 6        |
| 7.2      | Measured behaviour of the packaged application . . . . .          | 6        |
| 7.3      | Cross-platform status . . . . .                                   | 6        |
| <b>8</b> | <b>Sprint II Outcome</b>                                          | <b>7</b> |
| 8.1      | Retrospective . . . . .                                           | 7        |

# 1 Sprint Overview

## 1.1 Period and scope

Sprint II ran from 7 July 2026 to 30 July 2026. Sprint I produced an application that worked on the machines it was written on. Sprint II addressed a different problem: making that application installable and correct on a machine the team has never seen.

| Attribute                            | Value                                                    |
|--------------------------------------|----------------------------------------------------------|
| Duration                             | 7 July 2026 to 30 July 2026                              |
| Active development days              | 12                                                       |
| Commits                              | 106                                                      |
| Pull requests merged (project total) | 42                                                       |
| Deliverable                          | signed, validated installers for three operating systems |

The team worked jointly, with interface work, backend correction, and release engineering proceeding in parallel and integrating through pull requests.

## 1.2 Sprint goal statement

**Goal.** A person who has never installed a developer tool should be able to download one file, run it, and have every feature work, including on a computer with no Python, no model runtime, and no LaTeX distribution, and with the network disconnected.

# 2 Design Problem: The Unprepared Machine

Sprint I made an implicit assumption that a developer machine and a user machine are alike. They are not, and every significant defect of this sprint came from that single assumption.

| Resource          | Developer machine                | User machine            |
|-------------------|----------------------------------|-------------------------|
| Python runtime    | installed, correct version       | absent                  |
| Model weights     | present in the working directory | absent                  |
| Embedding weights | cached from earlier downloads    | absent                  |
| TeX distribution  | installed                        | absent                  |
| Working directory | the project checkout             | arbitrary               |
| Network           | available                        | may be absent by design |

The design response was to make the installer self-contained, and then to build automation that proves it, because human inspection had already been shown to be unreliable.

# 3 Design of the Distribution System

## 3.1 Packaging architecture

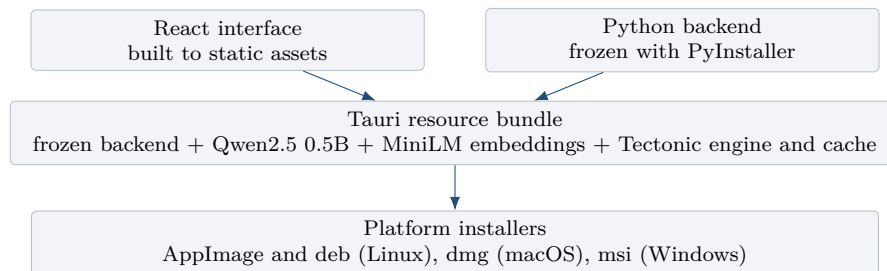


Figure 1: Packaging pipeline. Every runtime dependency is placed inside the resource bundle.

**Directory packaging rather than single-file.** PyInstaller was configured to emit a directory rather than a single executable. A single-file build extracts its entire payload to a temporary directory on every launch, which for a bundle approaching one gigabyte would impose that cost each time the application starts.

**Read-only and writable roots separated.** The bundle is read only once installed, so the design distinguishes the shipped payload from the user’s state. Model weights and the TeX cache are copied from the bundle into the user’s data directory on first use.

### 3.2 Release channel design

| Channel | Branch      | Tag form                   | Audience        |
|---------|-------------|----------------------------|-----------------|
| stable  | <b>main</b> | vX.Y.Z                     | the public      |
| beta    | <b>beta</b> | rolling tag <b>beta</b>    | invited testers |
| nightly | any         | rolling tag <b>nightly</b> | internal        |

Branches never produce installers. Only a tag does. A release therefore cannot occur accidentally through a merge, and the expensive three-platform build is paid only when it is intended.

### 3.3 Release guardrails

Each guardrail exists because the corresponding mistake is not reversible once an installer is published.

| Refusal condition            | Reason                                                  |
|------------------------------|---------------------------------------------------------|
| working tree not clean       | uncommitted work would not be in the build              |
| tag already exists           | re-tagging changes the meaning of a version silently    |
| version lower than published | the updater would move installed applications backwards |
| CI not green for that commit | a tag builds installers, so a red commit ships broken   |
| any asset near 2 GiB         | the hosting platform rejects it after a long build      |

## 4 Implementation

### 4.1 Phase 1: packaging and delivery, 7 to 24 July

- A public download page was produced and deployed.
- Cross-platform builds were automated for Linux, macOS and Windows from a single configuration file, so adding a platform is a data change rather than a workflow change.
- Hardware-aware model routing was introduced, selecting a model against currently available memory rather than installed memory, on the assumption that the user is running other software.
- In-application updates were implemented against signed release manifests, with per-channel endpoints so a beta installation updates within the beta channel.
- Linux packaging was moved to `appimagetool` after the default tool proved unable to resolve the libraries vendored inside the frozen Python bundle.

## 4.2 Phase 2: quality infrastructure, 25 to 29 July

- An automated test suite was introduced, growing to 309 tests across 23 modules.
- Hardware diagnosis was reimplemented in Rust inside the desktop shell. The previous implementation imported a deep learning framework purely to probe for a GPU, adding several seconds to every launch.
- A local verification gate was written that mirrors the continuous integration configuration exactly, so a developer sees the same result locally that the server will produce.
- Git hooks were installed from a version-controlled directory, so the whole team runs identical checks and updates arrive with a normal pull.
- Model discovery across runtimes was added, so weights already present through other local tools are reused rather than downloaded again. This required a canonical naming key, because the same weights are named differently by each runtime.

## 4.3 Phase 3: release, failure and correction, 29 to 30 July

Three releases were published. The third, version 1.0.0, did not work.

**Observed failure.** The published v1.0.0 installer displayed a loading screen indefinitely. Continuous integration was green throughout, because every test exercised the source tree and no test had ever launched the packaged application.

# 5 Root Cause Analysis

## 5.1 Why the application did not start

Three defects compounded, each concealing the next.

1. **The backend could not be located.** The packaging tool installs the executable into one directory and its resources into another. The lookup searched only beside the executable, so inside the installed application it found nothing and fell back to its development path: launching a system Python interpreter.
2. **The fallback interpreter was already broken.** The Linux application image exports an environment variable that redirects a Python interpreter's standard library search into the mounted image. A frozen interpreter ignores this; a system interpreter does not. It terminated before executing any project code.
3. **The failure was invisible.** The loading screen retried without limit and had no error state, so a backend that died in under a second was indistinguishable from one that was merely slow.

## 5.2 Further defects found in the same investigation

- The embedding weights were never placed in the bundle, so the first document ingested on an installed copy attempted a network download.
- The model directory was derived from the current working directory, which the application image changes before launching. The path resolved to a location that did not exist while the model sat inside the bundle. This was incorrect on every operating system for every installed copy.
- The window terminated roughly one second after appearing on the test system, caused by heap corruption in a graphics path of the system webview.

- The paper writer required a TeX distribution that no ordinary machine has, and was routed to two fine-tuned models that are never built, so it silently used the weakest model available.

### 5.3 Corrections applied

| Defect                   | Correction                                                                                                              |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------|
| backend not found        | resource location is obtained from the packaging framework, with explicit fallbacks per packaging format                |
| interpreter poisoned     | interpreter-controlling variables are removed from the child environment                                                |
| failure invisible        | startup is bounded, each step is reported with timings, and the trace is written to a log file that outlives the window |
| embedding weights absent | weights are staged into the bundle at build time                                                                        |
| model path from cwd      | the backend resolves its own paths; the shell no longer overrides them                                                  |
| window crash             | the failing renderer path is disabled on the affected platform                                                          |
| TeX absent               | a TeX engine and a warmed package cache are shipped in the installer                                                    |
| weak model for LaTeX     | routing prefers the larger model when it is present and fits memory                                                     |

## 6 Design of the Verification System

The corrective work was not limited to the defects. The failure demonstrated that testing source code does not establish that a product works, so four tiers of verification were designed.

| Tier                   | Frequency                    | What it establishes                                                                                                                        |
|------------------------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Local gate             | every push                   | lint, 309 tests, shell and workflow checks, before code leaves the machine                                                                 |
| Continuous integration | every push                   | the same checks on a clean machine                                                                                                         |
| Bundle validation      | every build, three platforms | the packaged application starts, resolves its models, ingests a document, retrieves, infers, and compiles a PDF                            |
| Interface smoke test   | every Linux build            | the launched application is still serving 45 seconds after it starts, which is the only way to observe a window that appears and then dies |

The third tier is decisive: continuous integration machines have no model and no TeX distribution installed, so a pass there is evidence about a clean user machine rather than about the build server.

### 6.1 Dependency audit

Rather than continue correcting defects as testers encountered them, every external dependency was enumerated by walking all imports and every subprocess invocation across 51 backend modules. The outcome was bounded and is recorded in the project documentation:

| Dependency                 | Required by       | Status                         |
|----------------------------|-------------------|--------------------------------|
| 15 Python packages         | backend           | frozen into the bundle         |
| Qwen2.5 0.5B weights       | inference         | shipped                        |
| MiniLM embedding weights   | ingestion, search | shipped                        |
| Tectonic and package cache | paper writer      | shipped                        |
| GPU query utility          | hardware probe    | optional, absence means no GPU |
| Windows task utility       | shutdown          | part of Windows                |

## 6.2 An instructive packaging defect

The first attempt to bundle the TeX engine appeared to succeed locally and failed on the Linux build server. Investigation established that the engine binary required a newer system C library than the build platform provides, so it never executed at all. The same binary would have been placed in the installer, which means users of long-term-support distributions would have received a paper writer that silently could not compile.

**Resolution.** The engine version was selected by measuring the minimum system library each release requires, rather than by taking the newest. The chosen version runs on the build platform and on older user systems, and reduced the bundled payload from 104 MB to 70 MB. The check is written into the build script so the next person does not repeat it.

## 7 Testing and Results

### 7.1 Automated verification

| Measure                           | Value       |
|-----------------------------------|-------------|
| Automated tests                   | 309         |
| Test modules                      | 23          |
| Test code                         | 2 709 lines |
| REST endpoints, contract-checked  | 36          |
| Platforms validated automatically | 3           |

An API contract test was added after a validation script was written against an endpoint that did not exist. The route table is defined in Python and the callers are written in JavaScript, so no type system spans the boundary. The test parses every route from the framework decorators and asserts that every caller targets one that exists.

### 7.2 Measured behaviour of the packaged application

Recorded on the built Linux installer: 15.3 GB memory, 16 threads, no usable GPU.

| Operation                                 | Time      |
|-------------------------------------------|-----------|
| Hardware diagnosis                        | 1 ms      |
| Backend ready                             | 1.75 s    |
| Document ingestion, first document        | 10.8 s    |
| Retrieval                                 | under 1 s |
| Chat response, model resident             | 1.2 s     |
| Summarisation on the base model           | 22 s      |
| LaTeX compilation with the bundled engine | 4.1 s     |

**Offline verification.** A document containing an equation, a formatted table and a plotted chart was compiled with outbound network sockets blocked at the interpreter level, so that a missing package could not be masked by a silent download. Compilation succeeded.

### 7.3 Cross-platform status

| Platform | Verification status                                                                                                                                                                                 |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Linux    | built, automatically validated, and exercised by hand on physical hardware                                                                                                                          |
| Windows  | built, automatically validated, and confirmed working by a human tester; not yet exercised across a range of machines                                                                               |
| macOS    | built and automatically validated; first-launch behaviour on physical hardware is obstructed by the operating system's handling of unsigned applications, so windowed operation remains unconfirmed |

## 8 Sprint II Outcome

| Objective                                               | Achieved |
|---------------------------------------------------------|----------|
| Installers for three operating systems                  | Yes      |
| Every runtime dependency shipped                        | Yes      |
| Application runs with no Python, model or TeX installed | Yes      |
| Automated regression suite                              | Yes      |
| Automated validation of the packaged product            | Yes      |
| Repeatable, guarded release process                     | Yes      |
| In-application updates                                  | Yes      |
| macOS windowed behaviour confirmed on hardware          | No       |
| Code signing for macOS and Windows                      | No       |

### 8.1 Retrospective

**What succeeded.** The release pipeline is configuration driven, so a platform or channel is a data change. The local gate mirrors the server exactly, which removed the class of failure where a developer is surprised by continuous integration. Recording decisions as they were taken produced 32 entries that made the dependency audit possible at all.

**What did not.** Three lessons are recorded honestly.

1. **The test suite verified the wrong artefact.** A complete set of passing tests coexisted with an installer that could not start.
2. **Defects were treated individually rather than by category.** The absent TeX engine, the absent embedding weights and the incorrect model path were one defect class, namely the assumption that a user machine resembles a developer machine. Each was found separately, by a person, after release.
3. **Diagnostic output was suppressed.** A build step that failed wrote its error to a null device and continued with a warning, which converted a five-minute diagnosis into a much longer one.

**Conclusion.** The technically demanding work was completed in Sprint I. What determined whether the software was usable by anyone else was packaging, dependency completeness, and verification of the delivered artefact, and that consumed the whole of Sprint II. The central lesson is that a feature which works only on the machine it was written on is not finished.